

1. 8-Puzzle Problem

AIM

To write a Python program to solve the 8-Puzzle problem using Breadth First Search (BFS).

PSEUDO CODE

1. Start
2. Define initial and goal states
3. Apply Breadth First Search (BFS)
4. Generate valid moves of the blank tile
5. Store visited states to avoid repetition
6. If goal state is found, display solution path
7. Stop

PROGRAM

```
from collections import deque

goal=((1,2,3),(4,5,6),(7,8,0))

def neighbors(state):
    s=[list(r) for r in state]
    for i in range(3):
        for j in range(3):
            if s[i][j]==0: x,y=i,j
    moves=[(-1,0),(1,0),(0,-1),(0,1)]
    for dx,dy in moves:
        nx,ny=x+dx,y+dy
        if 0<=nx<3 and 0<=ny<3:
            t=[r[:] for r in s]
            t[x][y],t[nx][ny]=t[nx][ny],t[x][y]
            yield tuple(map(tuple,t))

start=((1,2,3),(4,0,6),(7,5,8))
q=deque([(start,[])])
visited={start}
while q:
    state,path=q.popleft()
    if state==goal:
        print("Solved in",len(path),"moves")
        for st in path+[state]:
            for r in st: print(r)
```

```

        print()
    break
for n in neighbors(state):
    if n not in visited:
        visited.add(n)
        q.append((n,path+[state]))

```

INPUT

Initial State:

1 2 3

4 0 6

7 5 8

OUTPUT

Solved in 2 moves

(1,2,3)

(4,5,6)

(7,8,0)

RESULT

The Python program for solving the 8-Puzzle problem was implemented successfully and the goal state was obtained.

2. 8-Queen Problem

AIM

To write a Python program to solve the 8-Queen problem using Backtracking.

PSEUDO CODE

8. Start
9. Place a queen row by row
10. Check whether the position is safe
11. If safe, place the queen
12. If all queens are placed, print solution
13. Otherwise backtrack and try another position
14. Stop

PROGRAM

N=8

board=[-1]*N

```

def safe(r,c):
    for i in range(r):
        if board[i]==c or abs(board[i]-c)==abs(i-r):
            return False

```

```
        return True

def solve(r):
    if r==N:
        for i in range(N):
            line=['.']*N
            line[board[i]]='Q'
            print(' '.join(line))
        return True
    for c in range(N):
        if safe(r,c):
            board[r]=c
            if solve(r+1):
                return True
    return False

solve(0)
```

INPUT

No input required

OUTPUT

One valid arrangement of 8 queens is displayed on the board.

RESULT

The Python program for solving the 8-Queen problem using backtracking was executed successfully.